

Pagination in RESTful APIs | System Design Primer

 systemdesignschool.io/fundamentals/api-design-pagination



API Design - Pagination

When designing large-scale systems, you'll frequently encounter the challenge of returning large datasets through your APIs. Imagine you're building Twitter's feed API or Instagram's photo feed - you can't possibly return millions of records in a single request. This is where pagination comes in.

Understanding the Need for Pagination

Think about scrolling through your Instagram feed. Instead of loading thousands of photos at once, the app loads maybe 20-30 photos initially, then loads more as you scroll. This approach solves several critical problems in system design:

- Your server doesn't get overwhelmed processing massive result sets
- The client (mobile app or browser) doesn't crash trying to handle too much data at once
- Network bandwidth is used efficiently
- Users get a smooth, responsive experience

Let's explore how to implement this in practice.

Pagination Strategies

Offset-Based Pagination: The Simple Approach

This is the most intuitive way to implement pagination, similar to how you might think about pages in a book. Here's how it works:

```
GET /api/feed?limit=10&offset=20
```

When implementing Instagram's feed API, this would be like saying "give me 10 posts, starting after the first 20." The SQL might look something like:

```
SELECT * FROM posts ORDER BY created_at DESC LIMIT 10 OFFSET 20;
```

While this approach is straightforward, it has a significant drawback: imagine a user is on page 1,000 of their Instagram feed. The database would need to scan and skip through 10,000 rows before returning the next 10 posts. This is why Instagram doesn't actually use this approach for their feed.

Cursor-Based Pagination: The Scalable Solution

This is what you'll commonly see in modern APIs powering infinite scroll feeds. Instead of counting offsets, you keep track of where you are using a pointer (cursor):

```
GET /api/feed?limit=10&cursor=eyJpZCI6MTAwfQ==
```

How cursors work: The cursor is provided by the API in its response and represents the position of the last item returned. For the first request, you don't include a cursor. The API response includes a `next_cursor` that you use in your next request to get the next page of results. This creates a chain where each response provides the cursor for the next request.

The cursor is typically an encoded value representing the last item you've seen. For Instagram's feed, it might encode the timestamp and ID of the last post you viewed:

```
# Example cursor implementation def get_feed(limit=10, cursor=None): if cursor: last_post = decode_cursor(cursor) # e.g., {"timestamp": "2024-03-15", "id": 123} query = """ SELECT * FROM posts WHERE (created_at, id) < (:timestamp, :id) ORDER BY created_at DESC, id DESC LIMIT :limit """ else: query = "SELECT * FROM posts ORDER BY created_at DESC, id DESC LIMIT :limit" posts = execute_query(query) next_cursor = encode_cursor({ "timestamp": posts[-1].created_at, "id": posts[-1].id }) if posts else None return { "posts": posts, "pagination": {"next_cursor": next_cursor} }
```

This approach is much more efficient because we're using indexed fields to filter directly to where we need to be, rather than counting offsets.

Keyset Pagination: Time-Series Optimization

When dealing with time-series data (like a chat system or activity feed), you can optimize further by using timestamps directly:

```
GET /api/messages?limit=50&before=2024-03-15T00:00:00Z
```

This is particularly powerful when combined with database indexes:

```
CREATE INDEX idx_messages_timestamp ON messages(created_at DESC);
```

Real-World Considerations

When implementing pagination in a system design interview, consider these aspects:

1. **Consistency:** What happens if new items are added while someone is paginating?
Cursor-based pagination handles this elegantly by maintaining a consistent view.

2. **Performance:** Discuss how your pagination strategy scales with data size:

```
# Bad: Performance degrades with offset SELECT * FROM posts LIMIT 10 OFFSET 1000000 # Has to scan 1M rows # Good: Consistent performance with cursors SELECT * FROM posts WHERE id < :last_seen_id LIMIT 10 # Uses index
```

3. **Response Format:** Include metadata that helps clients:

```
{ "data": [ /* posts */ ], "pagination": { "next_cursor": "eyJpZCI6MTAwfQ==", "has_more": true } }
```

Making the Right Choice

In a system design interview, your choice of pagination strategy should align with your use case:

- Building an admin dashboard with page numbers? Offset pagination is fine.
- Designing the next Twitter feed? You'll want cursor-based pagination.
- Creating a chat system? Consider keyset pagination with timestamps.

Remember to explain your reasoning. For example: "I'm choosing cursor-based pagination for this social feed because we expect users to scroll through thousands of posts, and we need consistent performance regardless of how far they scroll."

The key is demonstrating that you understand the tradeoffs and can choose the right tool for the specific problem you're solving.